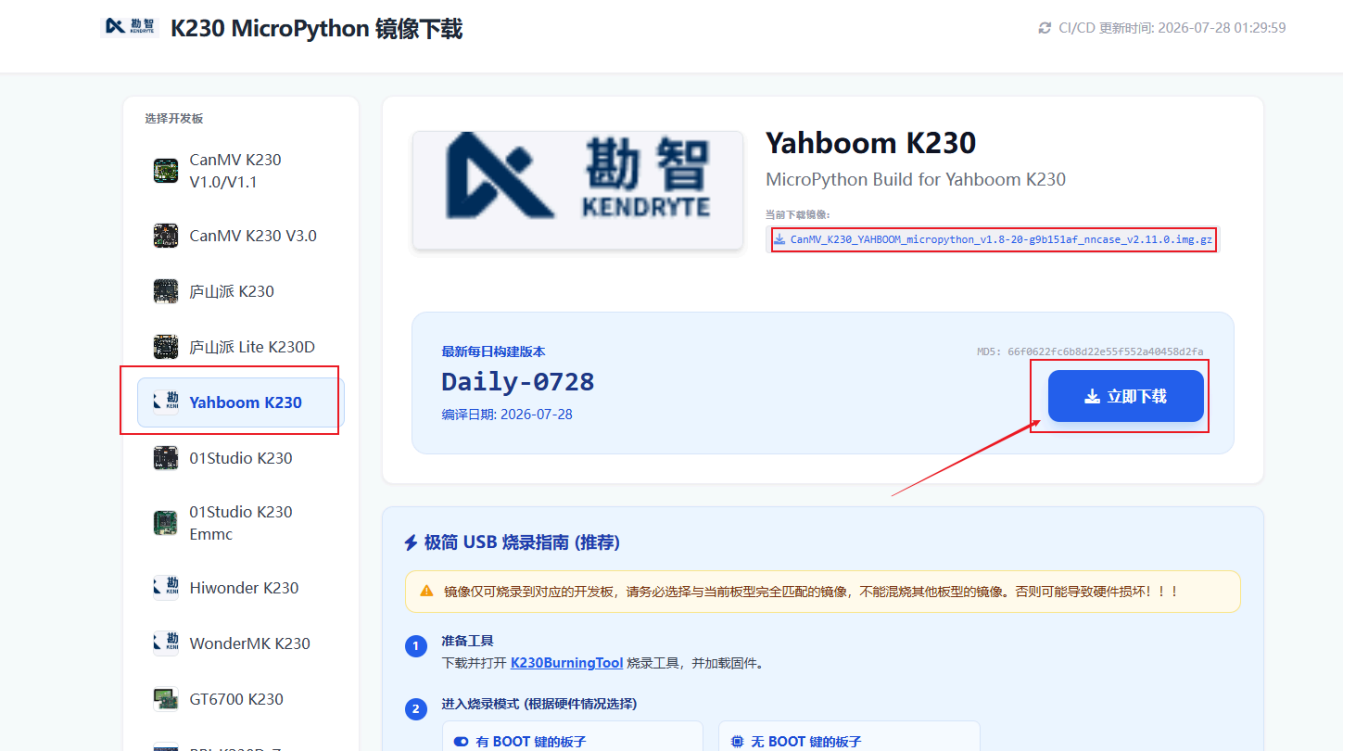
Drawing Functions

Preface

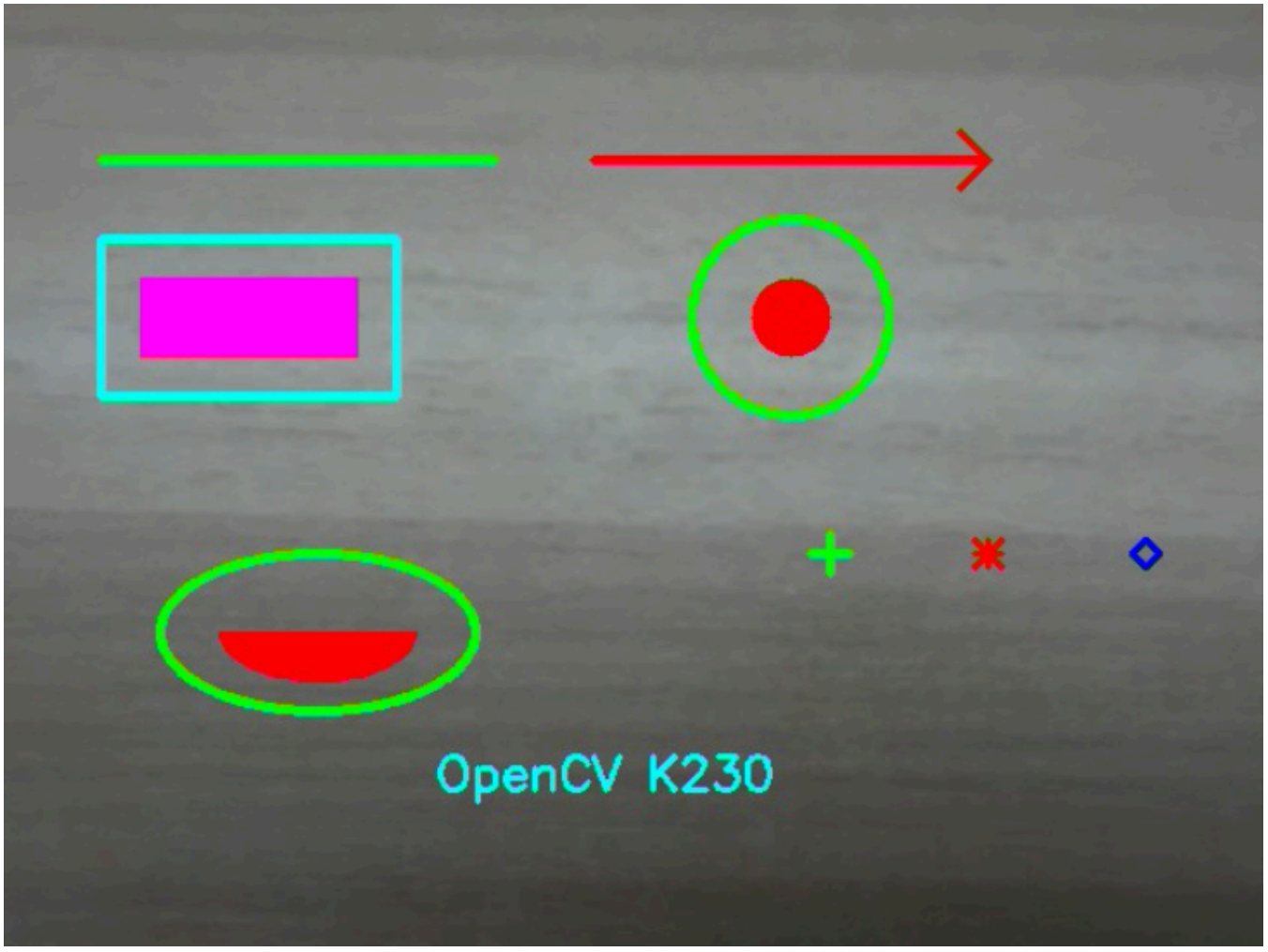
This tutorial is applicable to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. Download from Jianan official website https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/



Example Results

Run the example code for this section 08.camera_drawing.py.

This section draws lines, arrows, rectangles, circles, ellipses, text, marker symbols, and filled polygons on the real-time camera feed, for learning coordinate, color, and line width settings for detection result visualization.



Principle Explanation

Basic Parameters for OpenCV Drawing

Most drawing functions directly modify the input array. Common parameters include:

- **Coordinates**: Origin at top-left corner, x increases rightward, y increases downward.
- **Color**: Three-channel tuple, for example (0, 255, 0).
- **Line Width**: Positive number indicates contour width, -1 usually means filled.
- **Line Type**: LINE_AA generates anti-aliased edges but with slightly higher computation.

This example does not create a new output image, but directly overlays graphics on img_np, then displays the original img.

Code Overview

Initialize Camera and Display Module

```
W, H = 640, 480
sensor = Sensor(width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=W, height=H)
sensor.set_pixformat(Sensor.RGB888)
```

```
Display.init(Display.ST7701, width=640, height=480, to_ide=True)
sensor.run()
time.sleep(0.5)
```

Set Drawing Mode and Polygon Vertices

```
tri = np.array([
    [80.0, 60.0],
    [160.0, 60.0],
    [120.0, 120.0]
])
mode = 0
```

mode	Drawing Content
0	All graphics, source code default
1	Lines and arrows
2	Rectangles and circles
3	Ellipses and markers
4	Text and filled polygons

Draw Lines and Arrows

```
cv2.line(img_np, (50, 80), (250, 80), (0, 255, 0), 3)
cv2.arrowedLine(img_np, (300, 80), (500, 80), (255, 0, 0), 3)
```

line() takes start and end points, arrowedLine() adds an arrow at the end point.

Draw Rectangles and Circles

```
cv2.rectangle(img_np, (50, 120), (200, 200), (0, 255, 255), 3)
cv2.rectangle(img_np, (70, 140), (180, 180), (255, 0, 255), -1)
cv2.circle(img_np, (400, 160), 50, (0, 255, 0), 3)
cv2.circle(img_np, (400, 160), 20, (255, 0, 0), -1)
```

Line width -1 means filled. Rectangle parameters are top-left and bottom-right corners, circle parameters are center and radius.

Draw Ellipses and Markers

```
cv2.ellipse(img_np, (160, 320), (80, 40), 0, 0, 360, (0, 255, 0), 3)
cv2.ellipse(img_np, (160, 320), (50, 25), 0, 0, 180, (255, 0, 0), -1)

cv2.drawMarker(img_np, (420, 280), (0, 255, 0), cv2.MARKER_CROSS, 20, 3)
cv2.drawMarker(img_np, (500, 280), (255, 0, 0), cv2.MARKER_STAR, 15, 2)
cv2.drawMarker(img_np, (580, 280), (0, 0, 255), cv2.MARKER_DIAMOND, 15, 2)
```

Ellipse axis length parameters are semi-axis lengths. Angle range 0-360 draws a full ellipse, 0-180 draws a half ellipse.

Draw Text and Filled Polygons

```
cv2.putText(
    img_np, "OpenCV K230", (220, 400),
    cv2.FONT_HERSHEY_SIMPLEX, 0.8,
    (0, 255, 255), 2
)

cv2.fillPoly(
    img_np,
    [tri + np.array([[0.0, 420.0], [0.0, 420.0], [0.0, 420.0]])],
    (255, 100, 100),
    cv2.LINE_AA
)
```

In the source code, the triangle y-coordinates after translation are 480, 480, 540. Since these are at or below the bottom boundary of the 480-pixel high canvas, fillPoly() is likely invisible. This is a coordinate issue in the current example, not a function failure. To display the triangle, change the vertical offset from 420 to 300.

Resource Release

```
finally:
    sensor.stop()
    Display.deinit()
    os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
    time.sleep_ms(100)
```

Parameter Adjustment Notes

- **Coordinate Bounds**: x should be in 0-639, y should be in 0-479. Parts outside this region will be clipped.
- **Line Width**: Detection boxes commonly use 1-3; too thick will obscure target details.
- **Text Position**: The coordinates of putText() are the left end of the text baseline, not the top-left corner of the text box.
- **Color Order**: The current cv2 interface uses BGR color tuples as shown in the examples; when display results do not match, verify the firmware channel order with a solid color test.
- **Polygon Data Type**: The current example uses floating point arrays. If the firmware function has strict type requirements, change to an integer coordinate array that it supports.
- **Performance**: Large amounts of text and anti-aliased graphics increase per-frame overhead. In actual projects, only draw necessary information.